

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	CSA1205	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL2- DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:		Reg.No.:	
Department:		Date:	

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

ANS:

The problem: 8-bit signed addition

In an 8-bit signed 2's complement system, the range is:

-128 to +127

So:

- +120 is valid
- +50 is valid
- But $120 + 50 = 170$, which **cannot be represented** in 8-bit signed 2's complement.

Binary representation

+120:

120 = 01111000

+50:

50 = 00110010

Now add them:

01111000 (+120)

+ 00110010 (+50)

10101010

The result is:

10101010

Since the most significant bit is 1, this is interpreted as a **negative number** in signed 2's complement.

To find its value:

10101010

Invert the bits:

01010101

Add 1:

01010110

01010110 = 86

Therefore:

10101010 = -86

So the computer effectively gets:

120 + 50 = -86

This is **not an arithmetic error in the binary adder**. It is an **overflow** because 170 is outside the representable range of -128 to +127.

Program:

```
#include <stdio.h>
```

```
#include <stdint.h>
```

```
int main()
```

```
{
```

```
    int8_t a = 120;
```

```
    int8_t b = 50;
```

```
    int16_t result = (int16_t)a + (int16_t)b;
```

```
    printf("A = %d\n", a);
```

```
    printf("B = %d\n", b);
```

```
    printf("A + B = %d\n", result);
```

```
    if (result > 127 || result < -128)
```

```
    {
```

```
        printf("Overflow occurred!\n");
```

```
        printf("The result cannot be represented in 8-bit signed 2's complement.\n");
```

```
    }
```

```
    else
```

```
    {
```

```
        printf("No overflow occurred.\n");
```

```
    }
```

```
    return 0;
```

```
}
```

Output:

```
main.c Run Output
1 #include <stdio.h>
2 #include <stdint.h>
3
4 int main()
5 {
6     int8_t a = 120;
7     int8_t b = 50;
8
9     int16_t result = (int16_t)a + (int16_t)b;
10
11     printf("A = %d\n", a);
12     printf("B = %d\n", b);
13     printf("A + B = %d\n", result);
14
15     if (result > 127 || result < -128)
16     {
17         printf("Overflow occurred!\n");
18         printf("The result cannot be represented in 8-bit\n");
19         printf("signed 2's complement.\n");
20     }
21     else
22     {
23     }
```

A = 120
B = 50
A + B = 170
Overflow occurred!
The result cannot be represented in 8-bit signed 2's complement.

=== Code Execution Successful ===

2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.

ANS:

i) Problem with the Ripple Carry Adder

A **Ripple Carry Adder (RCA)** adds two binary numbers one bit at a time.

For an 8-bit addition:

A7 A6 A5 A4 A3 A2 A1 A0

B7 B6 B5 B4 B3 B2 B1 B0

The carry generated at one bit must travel to the **next bit**.

For example:

$C_0 \rightarrow FA_0 \rightarrow C_1 \rightarrow FA_1 \rightarrow C_2 \rightarrow FA_2 \rightarrow C_3 \rightarrow FA_3$

...

$\rightarrow FA_7 \rightarrow C_8$

This is why it is called a **ripple carry adder**: the carry "ripples" from the least significant bit toward the most significant bit.

ii). Example of the delay

Consider:

01111111
+ 00000001

10000000

Let's examine the carries.

Bit 0:

$1 + 1 = 0$, carry = 1

Bit 1:

$1 + 0 + \text{carry} = 0$, carry = 1

Bit 2:

$1 + 0 + \text{carry} = 0$, carry = 1

Bit 3:

$1 + 0 + \text{carry} = 0$, carry = 1

...

Bit 7:

$0 + 0 + \text{carry} = 1$

The carry has to propagate through almost **every bit**:

$C_0 \rightarrow C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow C_4 \rightarrow C_5 \rightarrow C_6 \rightarrow C_7 \rightarrow C_8$

Therefore, the higher-order sum bits have to wait for the carry from the previous stages.

For an n-bit ripple carry adder, the worst-case delay is approximately proportional to:

$O(n)$

So as the number of bits increases, the delay increases.

Program:

```
#include <stdio.h>
```

```
#include <stdint.h>
```

```
int main()
```

```
{
```

```
    uint8_t A = 0x7F; // 01111111
```

```
    uint8_t B = 0x01; // 00000001
```

```
    uint8_t sum = 0;
```

```
    int carry = 0;
```

```
    printf("A = ");
```

```
    for (int i = 7; i >= 0; i--)
```

```
        printf("%d", (A >> i) & 1);
```

```
    printf("\nB = ");
```

```
    for (int i = 7; i >= 0; i--)
```

```
        printf("%d", (B >> i) & 1);
```

```
    printf("\n\nCarry propagation:\n");
```

```
    for (int i = 0; i < 8; i++)
```

```
    {
```

```
        int a = (A >> i) & 1;
```

```
        int b = (B >> i) & 1;
```

```

int sum_bit = a ^ b ^ carry;

int new_carry = (a & b) | (carry & (a ^ b));

sum |= (sum_bit << i);

printf("Bit %d: A=%d B=%d Carry-in=%d Sum=%d Carry-out=%d\n",
      i, a, b, carry, sum_bit, new_carry);

carry = new_carry;
}

printf("\nResult = ");

for (int i = 7; i >= 0; i--)
    printf("%d", (sum >> i) & 1);

printf("\nFinal Carry = %d\n", carry);

return 0;
}

```

Output:

main.c	Run	Output
<pre> 1 #include <stdio.h> 2 #include <stdint.h> 3 4 int main() 5 { 6 uint8_t A = 0x7F; // 01111111 7 uint8_t B = 0x01; // 00000001 8 9 uint8_t sum = 0; 10 int carry = 0; 11 12 printf("A = "); 13 14 for (int i = 7; i >= 0; i--) 15 printf("%d", (A >> i) & 1); 16 17 printf("\nB = "); 18 19 for (int i = 7; i >= 0; i--) 20 printf("%d", (B >> i) & 1); 21 22 printf("\n\nCarry propagation:\n"); </pre>		<pre> A = 01111111 B = 00000001 Carry propagation: Bit 0: A=1 B=1 Carry-in=0 Sum=0 Carry-out=1 Bit 1: A=1 B=0 Carry-in=1 Sum=0 Carry-out=1 Bit 2: A=1 B=0 Carry-in=1 Sum=0 Carry-out=1 Bit 3: A=1 B=0 Carry-in=1 Sum=0 Carry-out=1 Bit 4: A=1 B=0 Carry-in=1 Sum=0 Carry-out=1 Bit 5: A=1 B=0 Carry-in=1 Sum=0 Carry-out=1 Bit 6: A=1 B=0 Carry-in=1 Sum=0 Carry-out=1 Bit 7: A=0 B=0 Carry-in=1 Sum=1 Carry-out=0 Result = 10000000 Final Carry = 0 === Code Execution Successful === </pre>

3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.

ANS:

Booth's Algorithm

Booth's algorithm is used to multiply **signed binary numbers** represented in 2's complement.

It uses three main registers:

- **M** → Multiplicand
- **Q** → Multiplier
- **A** → Accumulator
- **Q-1** → Extra bit used for checking the previous least-significant bit

Initially:

A = 0000

Q = Multiplier

M = Multiplicand

Q-1 = 0

At every iteration, we examine the pair:

(Q0, Q-1)

where Q0 is the least significant bit of Q.

The rules are:

Q0 Q-1 Operation

00 No operation

01 $A = A + M$

10 $A = A - M$

11 No operation

After the operation, perform an **arithmetic right shift** on:

A, Q, Q-1

The process is repeated once for every bit of the multiplier.

2. Why negative operands cause problems

Booth's algorithm depends heavily on **2's complement representation**.

For example, using 4 bits:

+3 = 0011

But:

-3

is obtained by:

0011

invert

1100

+ 1

1101

Therefore:

-3 = 1101

The important point is that when shifting a negative number, we must preserve the sign bit.

For example:

1101 >> 1

An **arithmetic right shift** gives:

1110

not:

0110

The 1 on the left is copied to preserve the negative sign.

3. Step-by-step example

Let's multiply:

$(-3) \times (+2)$

Using 4-bit 2's complement:

$M = -3 = 1101$

$Q = +2 = 0010$

$A = 0000$

$Q-1 = 0$

We need 4 iterations.

Initial state

$A = 0000$

$Q = 0010$

$Q-1 = 0$

$M = 1101$

Iteration 1

Check:

$Q_0 Q-1 = 0\ 0$

Rule:

00 → No operation

Now perform arithmetic right shift:

A Q Q-1

0000 0010 0

↓

0000 0001 0

State:

A = 0000

Q = 0001

Q-1 = 0

Iteration 2

Check:

Q0 Q-1 = 1 0

Rule:

10 → $A = A - M$

Since:

$M = 1101 = -3$

we calculate:

A - M

= $0 - (-3)$

= +3

Binary:

0000

- 1101

0011

So:

A = 0011

Now arithmetic right shift:

A Q Q-1

0011 0001 0

becomes:

0001 1000 1

Therefore:

A = 0001

Q = 1000

Q-1 = 1

Iteration 3

Check:

Q0 Q-1 = 0 1

Rule:

01 $\rightarrow$ A = A + M

Therefore:

0001

+ 1101

1110

1110 represents -2.

Now arithmetic right shift:

A Q Q-1

1110 1000 1

becomes:

1111 0100 0

So:

A = 1111

Q = 0100

Q-1 = 0

Notice that the leftmost 1 is preserved. This is **sign extension**.

Iteration 4

Check:

Q0 Q-1 = 0 0

Rule:

00 → No operation

Arithmetic right shift:

1111 0100 0

becomes:

1111 1010 0

Final result is:

A Q = 11111010

11111010 is an 8-bit 2's complement number.

Its value is:

11111010 = -6

Therefore:

$(-3) \times (+2) = -6$

Correct.

Program:

```
#include <stdio.h>
```

```
#include <stdint.h>
```

```

void printBinary(int value, int bits)
{
    for (int i = bits - 1; i >= 0; i--)
    {
        printf("%d", (value >> i) & 1);
    }
}

```

```

int main()
{
    int8_t multiplicand = -3;
    int8_t multiplier = 2;

    int16_t A = 0;
    int16_t M = multiplicand;
    int16_t Q = multiplier;
    int Q_minus_1 = 0;

    int n = 8;

    printf("Multiplicand = %d\n", multiplicand);
    printf("Multiplier = %d\n\n", multiplier);

    for (int i = 0; i < n; i++)
    {
        int Q0 = Q & 1;

        printf("Iteration %d\n", i + 1);
    }
}

```

```

printf("A = ");
printBinary(A & 0xFF, 8);

printf(" Q = ");
printBinary(Q & 0xFF, 8);

printf(" Q-1 = %d\n", Q_minus_1);

printf("Pair Q0,Q-1 = %d%d\n", Q0, Q_minus_1);

// Booth operation
if (Q0 == 0 && Q_minus_1 == 1)
{
    printf("Operation: A = A + M\n");
    A = A + M;
}
else if (Q0 == 1 && Q_minus_1 == 0)
{
    printf("Operation: A = A - M\n");
    A = A - M;
}
else
{
    printf("Operation: No operation\n");
}

// Save Q0 before shifting

```

```
int new_Q_minus_1 = Q0;
```

```
/*
```

```
 * Arithmetic right shift:
```

```
 * Shift A right while preserving its sign bit.
```

```
*/
```

```
int A_sign = A & 0x80;
```

```
Q = (Q >> 1) | ((A & 1) << 7);
```

```
A = A >> 1;
```

```
if (A_sign)
```

```
{
```

```
    A = A | 0x80;
```

```
}
```

```
Q_minus_1 = new_Q_minus_1;
```

```
printf("After shift:\n");
```

```
printf("A = ");
```

```
printBinary(A & 0xFF, 8);
```

```
printf(" Q = ");
```

```
printBinary(Q & 0xFF, 8);
```

```
printf(" Q-1 = %d\n\n", Q_minus_1);
```



```

}

int16_t result = (A << 8) | (Q & 0xFF);

printf("Final Result = ");

printBinary(result & 0xFFFF, 16);

printf("\nDecimal Result = %d\n", result);

return 0;
}

```

Output:

main.c	Run	Output
<pre> #include <stdio.h> #include <stdint.h> void printBinary(int value, int bits) { for (int i = bits - 1; i >= 0; i--) { printf("%d", (value >> i) & 1); } } int main() { int8_t multiplicand = -3; int8_t multiplier = 2; int16_t A = 0; int16_t M = multiplicand; int16_t Q = multiplier; int Q_minus_1 = 0; int n = 8; </pre>		<pre> Multiplicand = -3 Multiplier = 2 Iteration 1 A = 00000000 Q = 00000010 Q-1 = 0 Pair Q0,Q-1 = 00 Operation: No operation After shift: A = 00000000 Q = 00000001 Q-1 = 0 Iteration 2 A = 00000000 Q = 00000001 Q-1 = 0 Pair Q0,Q-1 = 10 Operation: A = A - M After shift: A = 00000001 Q = 10000000 Q-1 = 1 Iteration 3 A = 00000001 Q = 10000000 Q-1 = 1 Pair Q0,Q-1 = 01 Operation: A = A + M After shift: </pre>

4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.

ANS:

Restoring Division Algorithm

The **restoring division algorithm** performs binary division by repeatedly:

1. Shift the partial remainder.
2. Subtract the divisor.
3. Check whether the result is negative.
4. If negative, **restore** the previous remainder by adding the divisor back.
5. Set the quotient bit accordingly.

The problem is step 4.

Whenever the subtraction produces a negative partial remainder, the processor has to perform an **extra addition** to restore the previous value.

For an n-bit division, this can result in many unnecessary add/subtract operations.

2. Basic workflow

For unsigned division, we can use:

A = 0 Partial remainder

Q = Dividend Quotient

M = Divisor

For each bit:

1. Shift A and Q left

2. $A = A - M$

3. If $A < 0$:

$A = A + M$ $\leftarrow$ RESTORE

Q0 = 0

Else:

Q0 = 1

The highlighted restoration is the performance bottleneck.

3. Example: $7 \div 3$

Let's use:

Dividend = 7 = 0111

Divisor = 3 = 0011

Expected answer:

$7 \div 3 = 2$ remainder 1

Initial state

A = 0000

Q = 0111

M = 0011

After shifting and subtracting, whenever A becomes negative, the restoring algorithm has to add M back.

Conceptually:

$A = A - M$

If:

$A < 0$

then:

$A = A + M$

That extra addition doesn't directly contribute to the final quotient. It is required because the restoring algorithm immediately restores the previous remainder.

This is the **unnecessary operation** causing the performance bottleneck.

Program:

```
#include <stdio.h>
```

```
#include <stdint.h>
```

```
void printBinary(uint16_t value, int bits)
```

```
{
```

```
    for (int i = bits - 1; i >= 0; i--)
```

```
    {
```

```
        printf("%d", (value >> i) & 1);
```

```
}  
}
```

```
int main()
```

```
{
```

```
    uint8_t dividend = 7;
```

```
    uint8_t divisor = 3;
```

```
    uint16_t A = 0;
```

```
    uint16_t Q = dividend;
```

```
    uint16_t M = divisor;
```

```
    int n = 4;
```

```
    int restoration_count = 0;
```

```
    printf("Dividend = %d\n", dividend);
```

```
    printf("Divisor = %d\n\n", divisor);
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        // Shift A and Q left together
```

```
        A = (A << 1) | ((Q >> (n - 1)) & 1);
```

```
        Q = (Q << 1) & 0x0F;
```

```
        printf("Iteration %d\n", i + 1);
```

```
        printf("After shift: A = ");
```

```
        printBinary(A, n + 1);
```

```

printf(", Q = ");
printBinary(Q, n);

printf("\n");

// Subtract divisor
A = A - M;

printf("After A-M: A = ");
printBinary(A, n + 1);
printf("\n");

/*
 * Check whether subtraction produced
 * a negative value.
 */
if (A & (1 << n))
{
    // Restore A
    A = A + M;
    restoration_count++;

    printf("Negative remainder -> RESTORE\n");

    // Quotient bit = 0
    Q = Q & 0xFE;
}
else

```

```

{
    // Quotient bit = 1
    Q = Q | 1;

    printf("Positive remainder -> No restore\n");
}

printf("\n");
}

printf("Quotient = %d\n", Q);
printf("Remainder = %d\n", A);
printf("Restorations performed = %d\n", restoration_count);

return 0;
}

```

Output:

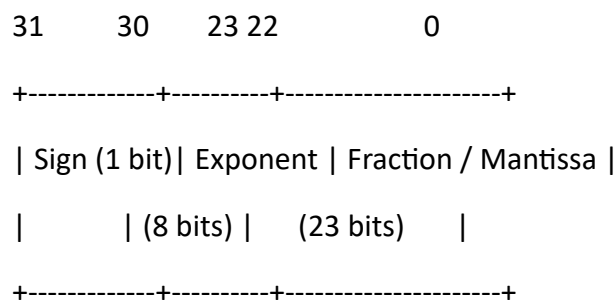
main.c	Run	Output
<pre> 1 #include <stdio.h> 2 #include <stdint.h> 3 4 void printBinary(uint16_t value, int bits) 5 { 6 for (int i = bits - 1; i >= 0; i--) 7 { 8 printf("%d", (value >> i) & 1); 9 } 10 } 11 12 int main() 13 { 14 uint8_t dividend = 7; 15 uint8_t divisor = 3; 16 17 uint16_t A = 0; 18 uint16_t Q = dividend; 19 uint16_t M = divisor; 20 21 int n = 4; 22 int restoration_count = 0; 23 </pre>		<pre> Dividend = 7 Divisor = 3 Iteration 1 After shift: A = 00000, Q = 1110 After A-M: A = 11101 Negative remainder -> RESTORE Iteration 2 After shift: A = 00001, Q = 1100 After A-M: A = 11110 Negative remainder -> RESTORE Iteration 3 After shift: A = 00011, Q = 1000 After A-M: A = 00000 Positive remainder -> No restore Iteration 4 After shift: A = 00001, Q = 0010 After A-M: A = 11110 Negative remainder -> RESTORE </pre>

5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Ans:

IEEE 754 Single Precision

A 32-bit IEEE 754 single-precision floating-point number has:



It consists of:

- **1 sign bit**
- **8 exponent bits**
- **23 fraction bits**
- Effective precision: approximately **24 significant binary bits** because of the implicit leading 1.

The approximate decimal precision is about **7 significant decimal digits**.

2. Why adding very large and very small numbers causes problems

Consider:

$100000000.0 + 1.0$

Mathematically:

$100000000 + 1 = 100000001$

But a float does not necessarily have enough precision to represent the difference between 100000000 and 100000001.

The large number dominates the small number.

This happens because the exponents must first be **aligned**.

3. Exponent alignment

Suppose we want to add two floating-point numbers:

$$1.010... \times 2^{10}$$

+

$$1.001... \times 2^2$$

Before addition, the exponents must be equal.

The smaller number is shifted:

$$1.001 \times 2^2$$

↓

$$0.00001001 \times 2^{10}$$

Now they can be added:

$$1.01000000 \times 2^{10}$$

+

$$0.00001001 \times 2^{10}$$

$$1.01001001 \times 2^{10}$$

The problem is that the smaller number may have to be shifted so far right that many of its significant bits are lost.

For an extremely large difference in magnitude, the small number can effectively disappear.

Program:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    float large = 100000000.0f;
```



```
float small = 1.0f;
```

```
float result = large + small;
```

```
printf("Large = %.10f\n", large);
```

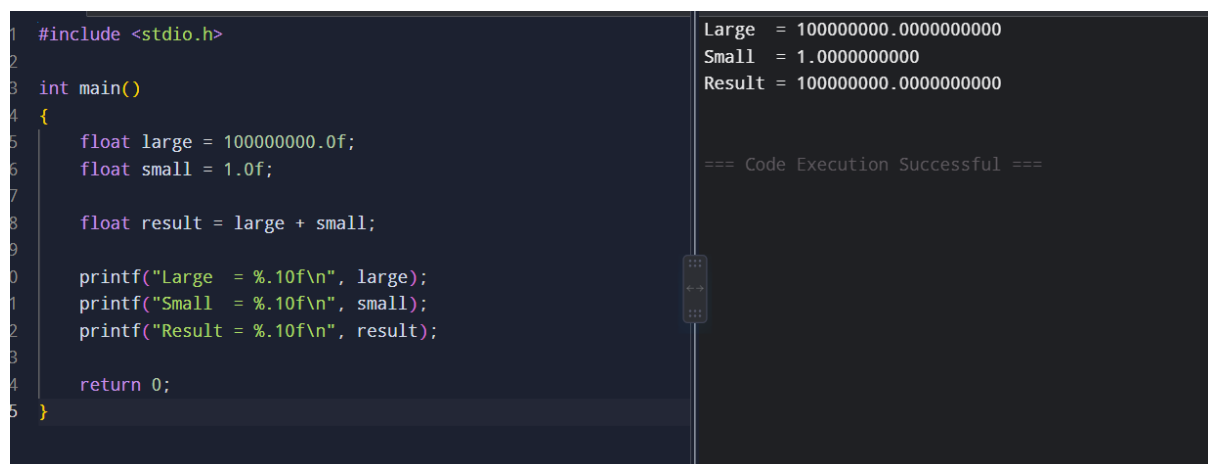
```
printf("Small = %.10f\n", small);
```

```
printf("Result = %.10f\n", result);
```

```
return 0;
```

```
}
```

Output:



```
1 #include <stdio.h>
2
3 int main()
4 {
5     float large = 100000000.0f;
6     float small = 1.0f;
7
8     float result = large + small;
9
10    printf("Large = %.10f\n", large);
11    printf("Small = %.10f\n", small);
12    printf("Result = %.10f\n", result);
13
14    return 0;
15 }
```

```
Large = 100000000.0000000000
Small = 1.0000000000
Result = 100000000.0000000000

=== Code Execution Successful ===
```

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
		and expected output			
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided